

Story 0: Merge Primitives (Foundation)

Epic: docs/project/epic-03-template-merge/epic.md **Tech Design:**
docs/project/epic-03-template-merge/tech-design.md

Objective

Establish the shared, testable primitives for template merge without changing any external behavior.

Scope

In Scope

- Merge field extraction primitive: `extractMergeFields(content) -> string[]`
- Merge operation primitive: `mergeContent(content, values) -> { content, mergeFields, unfilledFields }`
- Request/response schema shapes for merge (for downstream handlers/tools)

Out of Scope

- Any changes to prompt read responses (handled in Story 1)
- Any new endpoints/tools/UI (handled in later stories)

Dependencies / Prerequisites

- None

Acceptance Criteria

No epic ACs are closed by this foundation story directly.

Definition of Done

- [] The merge primitives exist in both runtimes that need them (Convex + service runtime)
- [] Unit tests cover parser rules from Epic Assumption A3 (valid identifiers only)

- [] Unit tests cover merge semantics (literal substitution, newline preservation, unfilled tracking, extra keys ignored)

Technical Implementation

Architecture Context

This story creates the two core primitives that all downstream stories depend on: a parser (`extractMergeFields`) and a merge utility (`mergeContent`). These are pure functions with no side effects, no I/O, and no dependencies on other modules.

Runtime boundary constraint: Convex runs in its own edge runtime. Imports between `convex/` and `src/` are not possible at runtime. This means the merge field regex must be duplicated in both runtimes. Both use the identical pattern; cross-runtime consistency is verified by shared test fixtures.

Modules and Responsibilities:

Module	Responsibility	Downstream Consumers
<code>convex/model/merge.ts</code>	<code>extractMergeFields()</code> — parser. Scans content for <code>{{fieldName}}</code> tokens, returns deduplicated field names in first-occurrence order.	<code>convex/model/prompts.ts</code> (Story 1: DTO enrichment on every read)
<code>src/lib/merge.ts</code>	<code>mergeContent()</code> — merge utility. Replaces <code>{{fieldName}}</code> tokens with values from a dictionary, tracks unfilled fields. Single-pass operation.	<code>src/routes/prompts.ts</code> (Story 2: REST endpoint), <code>src/lib/mcp.ts</code> (Story 3: MCP tool)

Module	Responsibility	Downstream Consumers
<code>src/schemas/prompts.ts</code>	<code>MergeRequestSchema</code> , <code>MergeResponseSchema</code> — Zod schemas for merge request/response shapes.	<code>src/routes/prompts.ts</code> (Story 2: request validation + response typing)
<code>tests/fixtures/merge.ts</code>	Shared test data used by parser tests, merge utility tests, and downstream endpoint/UI tests.	All test files across Stories 0-6

Types to Create

Parser: `convex/model/merge.ts`

```

/**
 * Regex for valid merge field syntax.
 * Matches {{fieldName}} where fieldName starts with [a-zA-Z_]
 * and contains only [a-zA-Z0-9_].
 * Does NOT match: {}, {{ name }}, {{foo.bar}}, {{my field}}
 *
 * NOTE: Duplicated in src/lib/merge.ts – Convex edge runtime boundary
 * prevents sharing imports between convex/ and src/ at runtime.
 */
const MERGE_FIELD_REGEX = /\{\{([a-zA-Z_][a-zA-Z0-9_]*)\}\}/g;

/**
 * Extract merge field names from prompt content.
 * Returns deduplicated array in first-occurrence order.
 *
 * @param content – Prompt content string (up to 100k chars)
 * @returns Deduplicated field names in first-occurrence order
 */
export function extractMergeFields(content: string): string[] {
  const seen = new Set<string>();
  const fields: string[] = [];
  let match: RegExpExecArray | null;

  // Reset lastIndex for safety (global regex)
  MERGE_FIELD_REGEX.lastIndex = 0;

  while ((match = MERGE_FIELD_REGEX.exec(content)) !== null) {
    const name = match[1];
    if (!seen.has(name)) {
      seen.add(name);
      fields.push(name);
    }
  }
}

```

```
    return fields;  
}
```

Merge Utility: `src/lib/merge.ts`

```

/**
 * NOTE: MERGE_FIELD_REGEX is duplicated from convex/model/merge.ts -
 * Convex edge runtime boundary prevents sharing imports between
 * convex/ and src/ at runtime.
 */
const MERGE_FIELD_REGEX = /\{\{([a-zA-Z_][a-zA-Z0-9_]*)\}\}/g;

export interface MergeResult {
  /** Merged content with values substituted */
  content: string;
  /** All merge fields found in the template (first-occurrence order) */
  mergeFields: string[];
  /** Fields with no matching value in the dictionary (first-occurrence order) */
  unfilledFields: string[];
}

/**
 * Replace {{fieldName}} tokens in content with values from dictionary.
 * Unfilled fields remain as {{fieldName}} in the output.
 * Values are substituted literally – no recursive processing.
 *
 * Implementation notes:
 * - Uses replace callback (not string replacement pattern) to avoid
 *   issues with special replacement patterns ($1, $&, etc.) in values.
 * - Uses Object.hasOwn() instead of `in` for value lookups to avoid
 *   prototype pollution (e.g., {{constructor}} would match
 *   Object.prototype
 *   with `in`).
 *
 * @param content – Template content with {{field}} tokens
 * @param values – Dictionary of field name -> replacement value
 * @returns Merged content, all fields, and unfilled fields
 */

```

```

export function mergeContent(
  content: string,
  values: Record<string, string>,
): MergeResult {
  const seen = new Set<string>();
  const mergeFields: string[] = [];
  const unfilledFields: string[] = [];

  // Single pass: replace fields and collect metadata simultaneously.
  const merged = content.replace(
    MERGE_FIELD_REGEX,
    (fullMatch, fieldName: string) => {
      if (!seen.has(fieldName)) {
        seen.add(fieldName);
        mergeFields.push(fieldName);
        if (!Object.hasOwn(values, fieldName)) {
          unfilledFields.push(fieldName);
        }
      }
      if (Object.hasOwn(values, fieldName)) {
        return values[fieldName];
      }
      return fullMatch; // Leave unfilled fields as-is
    },
  );

  return { content: merged, mergeFields, unfilledFields };
}

```

Zod Schemas: `src/schemas/prompts.ts`

```
// New: Merge request/response schemas
export const MergeRequestSchema = z.object({
  values: z.record(z.string(), z.string()),
});

export type MergeRequest = z.infer<typeof MergeRequestSchema>;

export const MergeResponseSchema = z.object({
  content: z.string(),
  mergeFields: z.array(z.string()),
  unfilledFields: z.array(z.string()),
});

export type MergeResponse = z.infer<typeof MergeResponseSchema>;
```

Test Fixtures

Shared merge test data in `tests/fixtures/merge.ts`, used by parser tests, merge utility tests, and downstream endpoint/UI tests:


```

export const MERGE_FIXTURES = {
  /** Content with two distinct fields */
  twoFields: {
    content: "Write {{code}} in {{language}}",
    mergeFields: ["code", "language"],
  },
  /** Content with duplicate field */
  duplicateField: {
    content: "{{language}} is great. I love {{language}}.",
    mergeFields: ["language"],
  },
  /** Content with no fields */
  noFields: {
    content: "Just a plain prompt with no merge fields.",
    mergeFields: [],
  },
  /** Empty content */
  emptyContent: {
    content: "",
    mergeFields: [],
  },
  /** Content with literal placeholder-like text (collision guard) */
  literalPlaceholder: {
    content: "Do not replace %%MERGE_0_name%% in output. Field: {{name}}",
    mergeFields: ["name"],
  },
  /** Content with edge cases */
  edgeCases: {
    content: "Valid: {{a}}, {{_b}}, {{c_1}}. Invalid: {{}}, {{ x }}, {{foo.bar}}",
    mergeFields: ["a", "_b", "c_1"],
  },
} as const;

```

Cross-function consistency: Both `extractMergeFields()` (Convex) and `mergeContent()` (API) use the same regex pattern (duplicated due to Convex runtime boundary) and return `mergeFields` in first-occurrence order. The parser tests and merge utility tests use the same `MERGE_FIXTURES` data, which implicitly verifies both functions produce identical `mergeFields` for the same content. If either regex is modified, both test suites will catch the drift.

Non-TC Decided Tests

Story 0 is a foundation story with no epic ACs — all tests are tech-design-decided:

Test File	Test Description	Source
tests/convex/prompts/mergeFields.test.ts	~8 parser tests (valid names, dedup, ordering, edge cases)	Tech Design §4. Parser Tests; early implementation of TC-1.1*, TC-1.2*, TC-1.4* contracts consumed by Story 1
tests/service/lib/merge.test.ts	~11 merge utility tests (substitution, unfilled tracking, no-op, literal values, prototype safety)	Tech Design §4. Merge Utility

Risks & Constraints

- **Regex drift between runtimes:** The regex in `convex/model/merge.ts` and `src/lib/merge.ts` must be identical. Both test suites use `MERGE_FIXTURES`, so drift would cause test failures, but the duplication is still the highest-risk seam in this story.
- **String.prototype.replace pitfalls:** `mergeContent` must use the callback form of `replace()`, not string replacement patterns. If a user value contains `$1`, `&$`, or `$$`, the string replacement form would interpret these as special patterns. The callback form substitutes literally.
- **Object.hasOwn vs in:** Value lookup must use `Object.hasOwn(values, fieldName)` to avoid prototype chain pollution. A field named `{{constructor}}` or `{{toString}}` would match `Object.prototype` with the `in` operator.

- **Global regex `lastIndex`** : `MERGE_FIELD_REGEX` is a global regex.
`extractMergeFields` must reset `lastIndex = 0` before use, or successive calls may produce different results.

Spec Deviation

None. Checked against Tech Design: §3. Low Altitude — Parser:

`convex/model/merge.ts` , §3. Low Altitude — Merge Utility: `src/lib/merge.ts` , §3.
 Low Altitude — Zod Schema Changes: `src/schemas/prompts.ts` , §5. Testing Strategy — Test Fixtures.

Technical Checklist

- ☐ Add `convex/model/merge.ts` with `extractMergeFields()` — strict identifier regex, `lastIndex` reset, Set -based deduplication, first-occurrence ordering.
 - ☐ Add `src/lib/merge.ts` with `mergeContent()` — callback-form `replace()` , `Object.hasOwn()` lookups, single-pass merge + metadata collection.
 - ☐ Add Zod schemas in `src/schemas/prompts.ts` : `MergeRequestSchema` , `MergeResponseSchema` , exported types.
 - ☐ Add fixtures in `tests/fixtures/merge.ts` (all six fixture entries from tech design).
 - ☐ Add parser unit tests in `tests/convex/prompts/mergeFields.test.ts` (~8 tests covering TC-1.1*, TC-1.2*, TC-1.4* parser rules).
 - ☐ Add merge unit tests in `tests/service/lib/merge.test.ts` (~11 tests: single/multi replacement, empty value, unfilled tracking, no-op, empty content, literal substitution, newlines, extra keys, prototype safety).
 - ☐ Verify: `bun run typecheck && bun run test` .
-